

Code Generation & Optimization Compilers

Consider the following IR code:

$$\begin{aligned}a &= b + c \\ b &= d + d \\ c &= b + c \\ d &= 2 * d\end{aligned}$$

1. What assembly code is generated for the above IR code, given 3 registers?
2. How is the above IR code optimized?
3. What assembly code is generated once the above IR code has been optimized?
4. What version is the most costly?
5. What does the dataflow analysis iterative algorithm return given the following dataflow graph:

$B1 :$	$d1 : a = b + c$
	$d2 : b = 2$
$B2 :$	$d3 : a = c + d$

and such that (1) B1 is the entry block; (2) B2 is the exit block; (3) B1 flows to B2; and (4) B2 flows to B1? What can we deduce from this analysis?

Solution

1. It can for example be translated as follows:

statement	instructions	register descriptors			address descriptors			
		R1	R2	R3	a	b	c	d
					a	b	c	d
$a = b + c$	LD R1, b							
	LD R2, c	b	c	a	R3	b, R1	c, R2	d
	ADD R3, R1, R2							
$b = d + d$	ST a, R3							
	LD R3, d	b	c	d	a	R1	c, R2	d, R3
	ADD R1, R3, R3							
$c = b + c$	ADD R2, R1, R2	b	c	d	a	R1	R2	d, R3
$d = 2 * d$	MUL R3, #2, R3	b	c	d	a	R1	R2	R3
end	ST b, R1							
	ST c, R2	b	c	d	a	b, R1	c, R2	d, R3
	ST d, R3							

2. Here is one way we can optimize the code:

- We can use the following law: $2 \times x = x + x$

$$\begin{aligned}
 a &= b + c \\
 b &= d + d \\
 c &= b + c \\
 d &= d + d
 \end{aligned}$$

- We can then use local common subexpression: $d + d$ is done twice

$$\begin{aligned}
 a &= b + c \\
 b &= d + d \\
 c &= b + c \\
 d &= b
 \end{aligned}$$

3. Let us first generate some code for the optimized version:

statement	instructions	register descriptors			address descriptors			
		R1	R2	R3	a	b	c	d
					a	b	c	d
$a = b + c$	LD R1, b							
	LD R2, c	b	c	a	R3	b, R1	c, R2	d
	ADD R3, R1, R2							
$b = d + d$	ST a, R3							
	LD R3, d	b	c	d	a	R1	c, R2	d, R3
	ADD R1, R3, R3							
$c = b + c$	ADD R2, R1, R2	b	c	d	a	R1	R2	d, R3
$d = b$		b, d	c		a	R1	R2	R1
end	ST b, R1							
	ST c, R2	b, d	c		a	b, R1	c, R2	d, R1
	ST d, R1							

Note that now we can reuse $R2$ when handling $a = b + c$, because c is not reused later.

The cost of the code generated for the non-optimized version is $(2 + 2 + 1) + (2 + 2 + 1) + 1 + 2 + (2 + 2 + 2) = 19$; while the code generated for the optimized version is only $(2 + 2 + 1) + (2 + 2 + 1) + 1 + (2 + 2 + 2) = 17$. Therefore, the non-optimized version is the most costly.

4. Here is an execution of the dataflow analysis algorithm:

- initially $\text{OUT}[B1] = \emptyset$ and $\text{OUT}[B2] = \emptyset$
- $\text{IN}[B1] = \emptyset$, $\text{OUT}[B1] = \{d1, d2\}$, $\text{IN}[B2] = \emptyset$, and $\text{OUT}[B2] = \{d3\}$
- $\text{IN}[B1] = \{d3\}$, $\text{OUT}[B1] = \{d1, d2\}$, $\text{IN}[B2] = \{d1, d2\}$, and $\text{OUT}[B2] = \{d3, d2\}$
- $\text{IN}[B1] = \{d2, d3\}$, $\text{OUT}[B1] = \{d1, d2\}$, $\text{IN}[B2] = \{d1, d2\}$, and $\text{OUT}[B2] = \{d3, d2\}$

We can deduce from $d2 \in \text{OUT}[B2]$ that b is a constant when exiting the graph, which could allow us performing constant folding, if ' b ' is used in a later block.